От слов к смыслу

Векторизация текста: Bag of Words, TF-IDF, эмбеддинги и классификация

Где мы и куда идём

Уже знаем: кодировки, токен, словарь, one-hot. Его главные минусы — длинные векторы и нет смысла.

1

Подсчёт слов

Bag of Words и TF-IDF: вектор для документа

2

Токены

BPE: как резать слова на части

3

Смысл

Word2Vec, FastText, BERT: векторы со смыслом

4

Классификация

Наивный Байес: спам или не спам

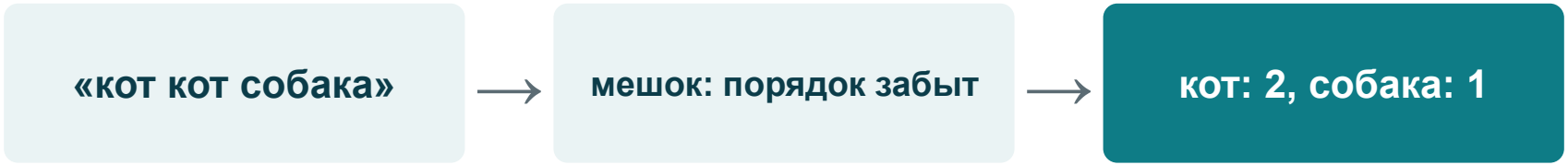
Как сравнить два текста



Нам нужен один вектор на весь документ.

Тогда тексты можно сравнивать, искать среди них и сортировать по темам.

Bag of Words: мешок слов



Как построить матрицу

1. Разбить каждый документ на слова
2. Собрать общий словарь
3. Посчитать слова в каждом документе
4. Записать счётчики в таблицу

Матрица «документ — слово»

	кот	пёс	ест	рыбу	кость	спит
кот ест рыбу	**1**	0	**1**	**1**	0	0
пёс ест кость	0	**1**	**1**	0	**1**	0
кот кот спит	**2**	0	0	0	0	**1**

Каждая строка — готовый вектор документа: «кот ест рыбу» = [1, 0, 1, 1, 0, 0].

Размер матрицы: число документов × размер словаря.

Минусы Bag of Words

Порядок потерян

«собака кусает человека» и «человек кусает собаку» дают один и тот же вектор.

Доминируют «и», «в», «на»

Служебные слова есть везде и получают самые большие числа, хотя смысла не несут.

Редкие слова — шум

Опечатки и редкие имена занимают место в словаре, но ничего не говорят о теме.

Частичное решение — n-граммы: считаем пары соседних слов, например «хорошая погода», как один признак. Цена — словарь сильно растёт.

Задание «Отзывы кафе» на Bag of Words

Вы владелец кафе и собрали 4 отзыва. Вы хотите понять, какие отзывы похожи по содержанию:
«суп вкусный суп горячий», «суп холодный официант медленный», «кофе вкусный кофе горячий», «кофе холодный кофе горький».

1. Токенизируйте каждый документ — разделите на слова.

2. Соберите уникальные токены из всех 4 документов и отсортируйте их по алфавиту.

3. Запишите словарь.

4. Для каждого отзыва посчитайте, сколько раз каждое слово из словаря в нём встречается. Заполните матрицу 8 x 4.

5. Посчитайте скалярное произведение для пар отзывов. По формуле:

$a \cdot b = a_1 \times b_1 + a_2 \times b_2 + \ldots + a_n \times b_n$

Заполните таблицу

Пара	Скалярное произведение
Отзыв1 · Отзыв2	
1 · 3	
1 · 4	
2 · 3	
2 · 4	
3 · 4	

1. Какие два документа самые похожие?
2. Какие два документа самые разные?
3. Отзыв1 и Отзыв2 — оба про суп. Отзыв1 и Отзыв3 — про разные блюда (суп vs кофе). Какая пара оказалась похожее? Почему?

Заполните таблицу

Пара документов	Скалярное произведение
Отзыв1 · Отзыв2	2
1 · 3	2
1 · 4	0
2 · 3	0
2 · 4	
3 · 4	4

1. Какие два документа самые похожие?
2. Какие два документа самые разные?
3. Отзыв1 и Отзыв2 — оба про суп. Отзыв1 и Отзыв3 — про разные блюда (суп vs кофе). Какая пара оказалась похожее? Почему?

Отсутствие семантики

- BoW-вектор не понимает, что «суп» и «кофе» — это подлежащее (предмет), а «вкусный» и «горячий» — прилагательные (оценка).
- Все слова равноправны и дают одинаковый вклад в сходство.
- Это и есть отсутствие семантики — то, что в следующем блоке будут решать TF-IDF (не полностью) и потом эмбеддинги (Word2Vec, BERT).

TF-IDF: важность слова

$$TF\text{-}IDF = TF \times IDF$$

TF — частота в документе

Доля слова среди всех слов документа. Чем чаще слово здесь, тем больше TF.

IDF — редкость в корпусе

IDF = ln(N / DF). Чем меньше документов содержат слово, тем больше IDF.

В корпусе из 1000 документов: IDF слова «и» ≈ 0.01, IDF слова «квантовый» ≈ 5.3.

TF-IDF на примере

Док. 1: «кот кот собака»

Док. 2: «собака собака птица»

N = 2 документа

Слово	TF (док. 1)	DF	IDF=ln(N/DF)	TF*IDF
кот	2/3	1	0.69	**0.46**
собака	1/3	2	0	**0**
птица	0	1	0.69	0

«собака» есть в обоих документах, она не помогает отличить док. 1 от док.2, поэтому вес 0.

TF-IDF на примере

Док. 1: «кот кот собака»

Док. 2: «собака собака птица»

N = 2 документа

Слово	TF (док. 2)	DF	IDF=ln(N/DF)	TF*IDF
кот	0	1	0.69	0
собака	2/3	2	0	**0**
птица	1/3	1	0.69	**0.23**

Главное слово документа 2 — это «птица» (TF-IDF = 0.23). Это единственное слово, которое отличает d2 от d1.

Автоматический фильтр стоп-слов

TF-IDF автоматически обнуляет слова, которые встречаются во всех документах (и, а, с, в и т.д.)
Уникальные слова получают наибольший вес

В TF-IDF используется натуральный логарифм. Повлияет ли использование десятичного логарифма на ранжирование документов?

Где TF-IDF не справляется

Синонимы

«хороший» и «отличный» — для него совсем разные слова.

Многозначность

«замок» (на двери) и «замок» (во дворце) получают один и тот же вес.

Новые слова

Слова, которых нет в словаре, в вектор не попадают.

Порядок слов

Как и BoW, TF-IDF его не учитывает.

Но как первый вариант TF-IDF отличный: быстрый, понятный, до сих пор используется в поиске.

Сравнение BoW и TF-IDF

Doc1: «кот кот собака» → **[2, 1, 0]** (кот=2, собака=1, птица=0)

Doc2: «собака собака птица» → **[0, 2, 1]**

d1 · d2 = 2×0 + 1×2 + 0×1 = 0 + 2 + 0 = 2

|d1| = √(2² + 1² + 0²) = √(4 + 1 + 0) = √5 ≈ 2.236|d2| = √(0² + 2² + 1²) = √(0 + 4 + 1) = √5 ≈ 2.236

cos(d1, d2) = 2 / (√5 × √5) = 2 / 5 = 0.4

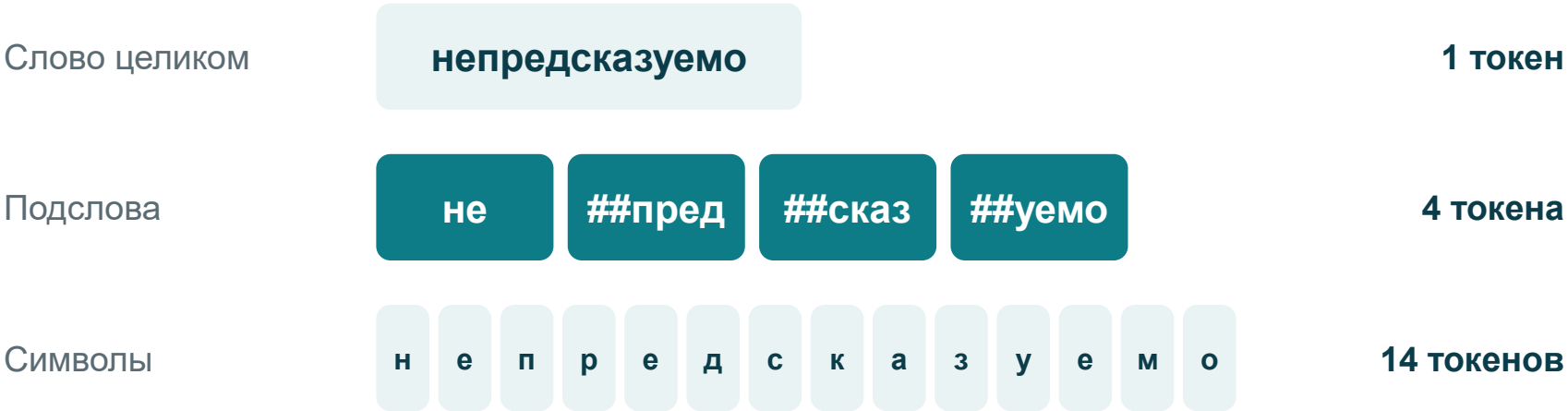
BoW cosine = 0.4 документы похожи

Doc1: **[0.46, 0, 0]** (кот=0.46, собака=0, птица=0)
Doc2: **[0, 0, 0.23]** (кот=0, собака=0, птица=0.23)
d1 · d2 = 0.46×0 + 0×0 + 0×0.23 = 0 + 0 + 0 = 0 – векторы не пересекаются

TF-IDFcosine = cos(d1, d2) = 0 документы разные

Токены: слово, часть слова, символ

Новое слово не из словаря превращается в <UNK>. Выход — резать слова на знакомые части.



Современные модели используют подслова — и никаких <UNK>.

BPE: склеиваем частые пары

Слова: lower, lowest, newest, widest

Старт: символы

l o w e r
l o w e s t
n e w e s t
w i d e s t

Шаг 1: e + s → es

l o w e r
l o w es t
n e w es t
w i d es t

Шаг 2: es + t → est

l o w e r
l o w est
n e w est
w i d est

Склейки: «lowest» = «low» + «est».
Финальный словарь = {l, o, w, e, s, t, n, i, d, es, est, lo, low}
9 символов + 4 слияния = 13 токенов

BPE: токенизация нового слова

Возьмём слово «lowness» — его не было в обучающем корпусе.

Применяем словарь от длинных токенов к коротким:

- Ищем **low** в начале — есть в словаре! → токен **low**
- Осталось: **er.** Ищем **er** — нет в словаре
- Разбиваем на символы: **e**, **r**

Итоговая токенизация: **lowness → [low, n, es, s] — 4 токена**

Без BPE: **[l, o, w, n, e, s, s]** — 7 токенов

Жадный алгоритм BPE

на каждом шаге берём самое частое слияние, не думая о «правильности» с лингвистической точки зрения. Алгоритм сам находит морфемы (**est**, **low**) и суффиксы через статистику. Это и делает его универсальным — работает для любого языка, любой морфологии.

WordPiece: токенизатор BERT

Подсловный токенизатор от Google. Используют: BERT, DistilBERT, ELECTRA, multilingual BERT. Словарь BERT-base: 30 522 токена.

	BPE	WordPiece
Что склеивает	самую частую пару: count(a, b)	пару с лучшим score = count(a, b) / (count(a) × count(b))
Метка продолжения	в базовом варианте нет	есть: ## у продолжения слова, а начало слова — без метки

Предпочитает пары, где оба куска редки сами по себе, но часто встречаются вместе.

Пример на учебном словаре



Задание: токенизация WordPiece

делаем руками

Словарь

Начало слова: play, work, hello, world, un

Продолжения: ##ing, ##s, ##er, ##ed, ##ment, ##ly, ##able, ##bel, ##ie, ##ful

Запасной вариант: любая буква a–z и ##a–##z

Правила

Берём самый длинный подходящий токен. Первый — без ##, остальные — с ##. Если ничего не подходит, берём одну букву.

Слово	Токены	Число
playing		
plays		
player		
workment		
playful		
unbelievable		

Подсказка: в последнем слове после каждого шага проверяйте, что осталось.

Ответ: токенизация WordPiece

Слово	Токены	Число
playing	play, ##ing	2
plays	play, ##s	2
player	play, ##er	2
workment	work, ##ment	2
playful	play, ##ful	2
unbelievable	**un, ##bel, ##ie, ##v, ##able**	**5**

Разбор: unbelievable

1. un, осталось believable
2. ##bel, осталось ievable
3. ##ie, осталось vable
4. ##v: токена ##vable нет, а ##able не подходит
5. ##able, готово

Много токенов на слово — признак того, что в словаре не хватает подходящих подслов.

Три популярных токенизатора

Метод	Модели	Словарь	Особенность
WordPiece	BERT	≈ 30 000	метка ## у продолжения слова
BPE (на байтах)	GPT-2, GPT-4	50–100 тыс.	работает с байтами UTF-8
SentencePiece	T5, LLaMA	≈ 32 000	не нужны пробелы между словами

Размер словаря — компромисс: маленький словарь даёт длинные цепочки токенов, большой — раздутую модель.

Токенизация vs эмбеддинг

	ДО ЭМБЕДДИНГОВ	С ЭМБЕДДИНГАМИ
Тип вектора	Разреженный (V-мерный, 99% нулей)	Плотный (300-3072d, все числа значимы)
Сёмантика	Нет («кот» ⊥ «собака»)	Есть («кот» ≈ «собака»)
Размер	Растёт со словарём (V=100K → 100K-мерные векторы)	Фиксированный (300 чисел на слово)
Обучение	Не нужно (просто считаем)	Нужно (нейросеть, корпус, часы

Слово узнают по компании

Кошка ест рыбу

Собака ест мясо

Лев ест антилопу

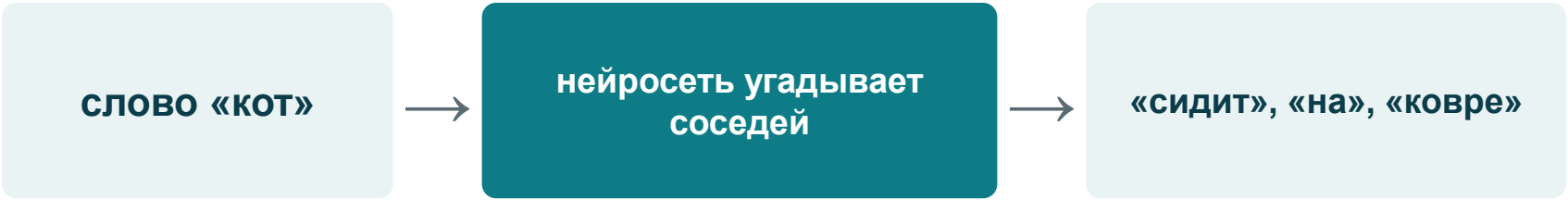
Кошка, собака и лев стоят в одном окружении — значит, их смыслы близки.

«Слово узнают по контексту, в котором оно встречается»

Джон Фёрт, 1957

На этой идее построены Word2Vec, GloVe, FastText и BERT.

Word2Vec: учим предсказывать соседей



Сама задача нам не нужна. Нужны числа внутри сети — после обучения они и есть векторы слов.

Вектор обычно из 100–300 чисел. Два режима обучения: CBOW — по соседям угадать слово, Skip-gram — по слову угадать соседей.

Задание Косинусное сходство слов

Какие слова ближе, а какие — дальше?

[0.90, 0.10, 0.00]

[0.85, 0.15, 0.05]

[0.70, 0.30, 0.10]

[0.60, 0.20, 0.00]

[0.00, 0.10, 0.95]

Арифметика со смыслом

король − мужчина + женщина	≈	королева
Париж − Франция + Италия	≈	Рим

Эти правила никто не закладывал: модель выучила их, читая огромные корпуса текстов.

Задание Векторные аналогии

- Для каждой аналогии запишите слово, которое должно получиться. Если ответ не приходит в голову — объясните, какие отношения тут зашифрованы. Формула: A − B + C = , где (A, B) — пара с отношением, C — начало новой пары

Аналогия	Отношение	Ответ
Минск − Беларусь + Россия	столица − страна	Москва
Лукашенко − Беларусь + Россия	президент − страна	Путин
ходить − ходил + бегать	инфинитив − прошедшее время	бегал

Выводы по Word2Vec

- выучивает смысл из контекста, без лингвиста
- Близость в векторном пространстве = семантическая близость
- Векторная арифметика (**king − man + woman = queen**) — удивительное свойство, которое показывает, что векторы кодируют отношения
- Минус: одно слово = один вектор (полисемия не различается)

GloVe и FastText

GloVe, 2014

Считает, как часто слова встречаются рядом, по всему корпусу сразу, и сжимает эту таблицу в короткие векторы.

Качество как у Word2Vec, обучается быстрее.

FastText, 2016

Слово = сумма векторов его кусочков из букв. Новое слово можно собрать из знакомых кусочков.

Хорош для русского: «делаю», «делает», «делали» получают близкие векторы.

Общий минус: одно слово — один вектор, контекст не учитывается.

Векторизатор Glove

- Дано: пара предложений: кот ест рыбу. собака ест мясо.
- Словарь (5 слов): {кот, собака, ест, рыбу, мясо}
- Смотрим только на соседей слева и справа и составляем пары слов.
- Соседние пары: (кот, ест) = 1 (ест, рыбу) = 1 (собака, ест) = 1 (ест, мясо) = 1
- Максимум у «ест» (1+1+0+1+1 = 4) — это «хаб» корпуса, встречается со всеми существительными.

- Никогда не встречаются рядом: «кот» ↔ «собака», «кот» ↔ «рыбу» (через ест), «рыбу» ↔ «мясо». GloVe заполнит эти нули через факторизацию матрицы, используя транзитивные связи (через «ест»).

Матрица конкуренций

	кот	собака	ест	рыбу	мясо
кот	0	0	1	0	0
собака	0	0	1	0	0
ест	1	1	0	1	1
рыбу	0	0	1	0	0
мясо	0	0	1	0	0

Пайплайн Glove

- Построение матрицы конкуренций X (V × V) по корпусу текстов
- Выбор размерности d (например, 100)
- Инициализация матриц W (V × d) и H (d × V) случайными числами
- Градиентный спуск: минимизируем $J = \Sigma f(X[i,j]) \times (w_i \cdot h_j - \log(X[i,j]+1))^2$
- Через 100-1000 эпох получаем обученные W и H
- Финальный эмбеддинг слова i = W[i] (или (W[i] + H[i])/2)
- Каждый токен имеет плотный вектор из d чисел

Проблема: один вектор на слово

«замок на двери сломался»

замок — устройство

«средневековый замок на холме»

замок — здание

У слова «замок» в обоих случаях один вектор.

Нужно, чтобы вектор менялся в зависимости от соседних слов. Это делают контекстные модели, например BERT.

BERT: вектор зависит от контекста

Как учится BERT

Кот [MASK] на ковре

↓

угадывает: «сидит»

Закрывают около 15% слов, и модель учится их восстанавливать.

2018 год выхода, Google

12 слоёв в BERT-base

768 чисел в векторе слова

110 млн параметров

BERT-векторизация в коде

3 строчки кода на Python

```
inputs = tokenizer(text,
return_tensors="pt")outputs =
model(**inputs)sentence_vector =
outputs.last_hidden_state[0].mean(dim=0)
```

Наивный Байес: спам или нет

Новое письмо: «бесплатно деньги»

Класс	Доля писем	«бесплатно»	«деньги»	Итог (перемножить)
спам	0.2	0.5	0.4	**0.04**
не спам	0.8	0.01	0.02	**0.00016**

Побеждает спам: его число в 250 раз больше.

«Наивный» — потому что считает слова независимыми друг от друга. Это неточно, но на практике работает.

Наивный Байес на практике

```
model = make_pipeline(
    TfidfVectorizer(), MultinomialNB())

model.fit(texts, labels)
model.predict(["бесплатно деньги"])
```

Три строки: TF-IDF превращает тексты в векторы, Байес их классифицирует.

Плюсы

очень быстрый и простой;
хватает небольшой выборки; даёт вероятности.

Минусы

не видит связей между словами;
плохо разделяет сложные темы.

Карта методов

Метод	Что даёт	Главный минус
One-hot	номер слова в векторе	огромный, без смысла
Bag of Words	вектор документа из счётчиков	нет порядка и важности
TF-IDF	веса: важные слова выше	нет смысла и синонимов
Word2Vec, FastText	вектор со смыслом	один вектор на слово
BERT	вектор с учётом контекста	дорогое обучение

Подслова (BPE) — общий шаг для современных моделей: слова режут на знакомые куски.

Что запомнить

1. BoW и TF-IDF превращают документ в вектор, считая слова. Просто, но без смысла.
2. Подслова (BPE) решают проблему новых слов.
3. Word2Vec и BERT дают векторы со смыслом, BERT — ещё и с учётом контекста.
4. Наивный Байес на TF-IDF — быстрый первый вариант для классификации текстов.

Читать дальше: Jurafsky & Martin, «Speech and Language Processing» (бесплатно онлайн); курс Hugging Face NLP.